

Short Paper

Autonomous Steering of Concentric Tube Robots via Nonlinear Model Predictive Control

Mohsen Khadem , John O'Neill , Zisos Mitros , Lyndon da Cruz, and Christos Bergeles 

Abstract—This article presents a model predictive controller (MPC) developed for the autonomous steering of concentric tube robots (CTRs). State-of-the-art CTR control relies on differential kinematics developed by local linearization of the CTRs mechanics model and cannot explicitly handle constraints on robot's joint limits or unstable configurations commonly known as snapping points. The proposed nonlinear MPC explicitly considers constraints on the robot configuration space (i.e., joint limits) and the robot's workspace (i.e., mixed boundary conditions on robot curvature). Additionally, the MPC calculates control decisions by optimizing the model-based predictions of future robot configurations. This way, it avoids configurations it cannot recover from, i.e., joint limits, singular configurations, and snapping. The proposed controller is evaluated via simulations and experimental studies with a variety of trajectories of increasing complexity. Simulation results demonstrate the capability of MPC to avoid singularities while satisfying robot mechanical constraints. Experimental results demonstrate that our solution enables following of trajectories unattainable by state-of-the-art controllers with mean error corresponding to 1% of robot arclength.

Index Terms—Continuum surgical robots, model predictive control.

I. INTRODUCTION

A concentric tube robot (CTR) is a continuum robot comprising a series of precurved elastic tubes. Each tube can be axially translated and rotated with respect to the others to control the robot shape and tip pose. CTRs can traverse confined spaces and manipulate objects in complex environments, similar to complementary surgical continuum robots. Deployment of CTRs inside the body requires precise control of robot motion.

The kinematic model that is widely used to control the CTR was derived in [1] and [2]. The model estimates the robot shape as a function of tubes' base translation, rotation, and known external forces, via a set of differential equations with boundary conditions split between the base and tip of the robot. Iterative numerical approaches such as shooting methods can solve the boundary value problem (BVP) and estimate the robot shape [1], [2].

Dupont *et al.* [1] used Fourier series to approximate the inverse kinematics of the robot for open-loop control. Gilbert *et al.* [3] proposed

a follow-the-leader approach by estimating deployment sequences that would lead the robot body to follow the position of its tip. Using these approaches, CTRs have been successfully employed with open-loop control in several clinical scenarios [4], [5], in which an operator directly controls the robot tip. However, in general, the challenges in introducing the robot up to the location of interest are purposefully kept minimal. To enable automatic deployment of CTRs through complex tortuous anatomy, accurate closed-loop control of the CTRs end-effector is essential. Then, assuming follow-the-leader architectures [3], [6] and safe intraluminal deployment, the shape will also conform to anatomical constraints.

Due to the complexity of the BVP model of CTR, calculating the inverse kinematics of the robot requires tailored computationally efficient methodologies. A straightforward approach is to approximate the Jacobian of the robot by finite differences on the solution of the BVP problem, e.g., [7]. Rucker *et al.* [8] proposed a numerical approach to increase the computational efficiency of Jacobian estimation for a CTR under external loading. Xu *et al.* [9] proposed real-time estimation of the robot Jacobian by discretizing the robot into small sublinks. The Jacobian was used for closed-loop control of the CTRs tip. Wu *et al.* [10] presented a model-less method for real-time estimation of the robot Jacobian via visual servoing. The Jacobian was then used to control the position of the CTR. Kudryavtsev *et al.* [11] developed a CTR with an embedded camera at the robot's tip. They used the linear approximation of the CTR Jacobian and visual feedback for real-time steering.

Most recent research in CTRs has focused on their open-loop steering [1], [3]. State-of-the-art closed-loop control strategies rely on linear approximation of robot inverse kinematics near equilibriums [9], [12] but neglect the system's nonlinearities, potentially leading to instability and reduced controller accuracy. Recently, researchers proposed application of dynamic active constraints that could improve the safety and stability also of CTRs [13].

It should be noted that the CTRs workspace is constrained by the tubes' precurvature, range of linear translation, and nonholonomic path-dependent constraints on robot motion in terms of tubes' end curvatures. These constraints make the tip trajectory and joint values dependent on their traversed path [12], [14]. Therefore, control methods that only use contemporary information may lead the robot to instabilities or configurations that it cannot recover from (see Fig. 1).

In this article, we design a novel controller for steering of CTRs end-effector. Unlike current CTR controllers [9], [11] that employ the local robot Jacobian, the model predictive controller (MPC) uses future predictions of robot behavior to handle explicit constraints on the robot state and control inputs, therefore being able to steer the robot away from unstable configurations commonly known as "snapping" points. Snapping is a mechanical instability in CTRs caused by the release of the elastic potential energy that is accumulated due to tube bending and twisting [1].

Fig. 1 shows a comparison of the MPC controller proposed in this article with state-of-the-art Jacobian-based controllers. The simulation results show that local controllers (e.g., Jacobian-based or damped-least-squares inverse kinematics) blindly steer the robot in configurations it cannot recover from, whereas the proposed MPC incorporates future

Manuscript received November 5, 2019; revised February 18, 2020; accepted April 26, 2020. Date of publication May 21, 2020; date of current version October 1, 2020. This work was supported by an ERC Starting Grant 714562. This paper was recommended for publication by Associate Editor H. Wang and Editor M. Yim upon evaluation of the reviewers' comments. (Lyndon da Cruz and Christos Bergeles contributed equally to this work.) (Corresponding author: Mohsen Khadem.)

Mohsen Khadem is with the School of Informatics, University of Edinburgh, Edinburgh EH8 9YL, U.K. (e-mail: mohsen.khadem@ed.ac.uk).

John O'Neill, Zisos Mitros, and Christos Bergeles are with the School of Biomedical Engineering and Imaging Sciences, King's College London, London WC2R 2LS, U.K. (e-mail: oneil463@umn.edu; zmitros@gmail.com; christos.bergeles@kcl.ac.uk).

Lyndon da Cruz is with the Moorfields Eye Hospital, London EC1V 2PD, U.K. (e-mail: l.cruz@ucl.ac.uk).

Color versions of one or more of the figures in this article are available online at <https://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TRO.2020.2991651

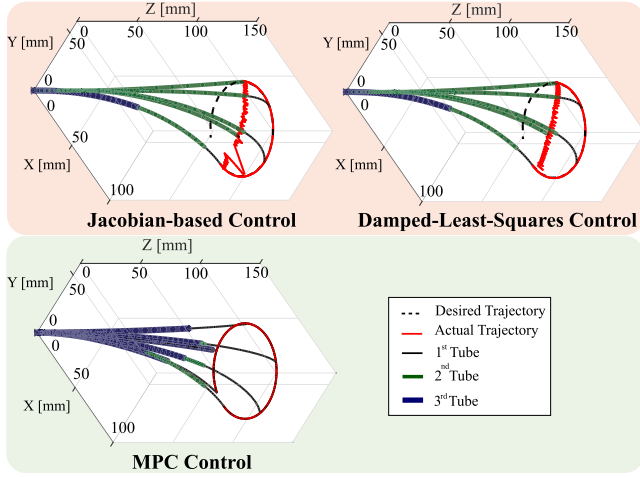


Fig. 1. Comparison of the designed controller with the state of the art.

knowledge to follow optimal paths. The proposed controller fills a gap between the locally intelligent and rapid Jacobian-based controllers, and the globally intelligent but computation-heavy path-planners [7], [15]. Additionally, the proposed MPC is designed such that it only requires the solution of the CTR model as an initial value problem (IVP) rather than a BVP. The controller includes the mixed boundary conditions as constraints on the optimal solution. Implementing only the solution of the IVP problem improves computational efficiency of control strategies. We make our software available online.¹

II. REVIEW OF CTR MECHANICS MODEL

CTR robot behavior is captured well by quasi-static models [1], [2]. Each concentric tube is modeled as a deformable curve with a Bishop frame attached to every point along its arclength, with the z -axis of the frame remaining tangent to the curve. The configuration of the robot can be defined using a unique set of 3-D centroids, $\mathbf{r}(s, t) : [0, \ell] \times [0, \infty] \rightarrow \mathbb{R}^3 \times [0, \infty]$, and a family of orthogonal transformations, $\mathbf{R}(s, t) : [0, \ell] \times [0, \infty] \rightarrow SO(3) \times [0, \infty]$. The spatial evolution of the curvature of the robot can be defined as $\mathbf{u}(s, t) = (\mathbf{R}^T(s, t)\mathbf{R}'(s, t))^\vee$, where the $\vee$ operator converts a skew-symmetric cross-product matrix to a vector in $\mathbb{R}^3$. Now, assuming the tubes are made of linear elastic isotropic materials without pretwist, and following the approach introduced in [1] and [2], we can derive the constitutive equations for calculating the instantaneous curvature of the tubes and the overall robot shape.

First, the robot is separated into transition points at which the continuity of shape and internal moment is enforced. Each segment contains up to N tubes. Example transition points are indicated by dashed lines in Fig. 2. Next, we consider that the final deformed curve of all tubes at a given time t must be equal to the curve of the innermost tube, i.e., $\mathbf{r}^i(s, t) = \mathbf{r}^1(s, t)$, as the inner most one has the maximum extension (following the guidelines introduced in [1]). We use $\alpha^i(s, t)$ to parameterize the tubes' twist angle around z -axis, i.e., $\mathbf{R}^i(s, t) = \mathbf{R}^1(s, t)\mathbf{R}_z(\alpha^i(s, t))$, where $\mathbf{R}_z$ denotes a rotation around the z -axis of the i th tube due to twist. Finally, based on these assumptions, tube curvatures are calculated by

$$\mathbf{r}^{1'}(s, t) = \mathbf{R}^1(s, t)\mathbf{e}_3 \quad (1a)$$

$$\mathbf{R}^{1'}(s, t) = \mathbf{R}^1(s, t)\hat{\mathbf{u}}^1(s, t) \quad (1b)$$

¹[Online]. Available: <http://www.github.com/RViMLab/TRO2020-CTR-Model-Predictive-Control>

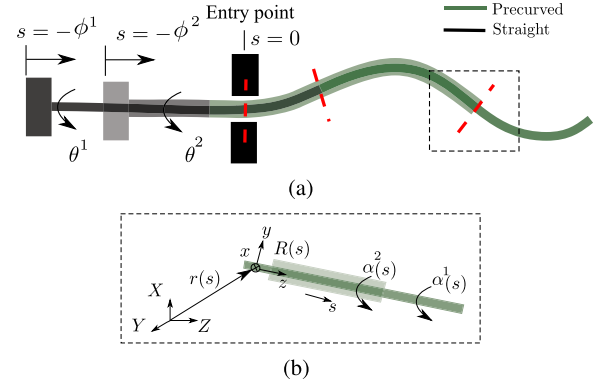


Fig. 2. Illustration of a CTR. Tubes are grasped at their proximal ends. The actuation variables $\theta^i(t)$ and $\phi^i(t)$ denote the proximal base rotation, and translation of the i th tube, respectively. Each tube comprises a straight and a curved part. $\alpha^i(s)$ denotes angular displacement of tube i at arclength s . Arc length is the length of the robot measured from the entry point, i.e., $s = 0$.

$$u_n^i(s, t) = \left(\sum_{j=1}^N \mathbf{K}^j \right)^{-1} \mathbf{R}_z^T(\alpha^i(s, t)) \quad (1c)$$

$$\times \left(\sum_{j=1}^N \mathbf{R}_z(\alpha^j(s, t)) \mathbf{K}^j \mathbf{U}^j \right) \Big|_{n=1,2}$$

$$u_3^{i'}(s, t) = \frac{E^i I^i}{G^i J^i} (u_1^i(s, t) U_2^i - u_2^i(s, t) U_1^i) \quad (1d)$$

$$\alpha^{i'}(s, t) = u_3^i(s, t) - u_3^1(s, t) \quad (1e)$$

where superscript $i = 1, \dots, N$ denotes the i th tube, with $i = 1$ corresponding to the innermost tube; subscript $n = 1, 2, 3$ denotes the n th element of a vector; $\mathbf{e}_3 = [0, 0, 1]^T$ is the unit vector aligned with the z -axis of the global coordinate frame; $\mathbf{U}^i$ denotes the precurvature of each tube in its reference configuration; $\alpha^{i'} = u_3^i$ denotes the angle of twist about the local z -axis with respect to the global frame; $\mathbf{K}^i = \text{diag}(E^i I^i, E^i I^i, G^i J^i)$ is the stiffness matrix for tube i ; E is the tube's Young's modulus; I is the second moment of inertia; G is the shear modulus; J is the polar moment of inertia.

In the absence of external forces/torques, the boundary conditions are specified in terms of tubes curvatures and actuators values by

$$\mathbf{r}^1(0, t) = [0 \ 0 \ 0]^T \quad (2a)$$

$$\mathbf{R}^1(0, t) = \mathbf{R}_z(\theta^1(t) - \phi^1(t)u_3^1(0, t)) \quad (2b)$$

$$\alpha^i(0, t) = \theta^i(t) - \phi^i(t)u_3^i(0, t) \quad (2c)$$

$$u_3^i(\ell^i + \phi^i(t), t) = U_3^i. \quad (2d)$$

The boundary conditions given in (2) define $\mathbf{r}$, $\mathbf{R}$, and α at the base of the robot, and curvatures along the z -direction, u_3^i , at the end of the tubes, thus forming a BVP. The mixture of boundary values imposes nonholonomic constraints on the motion of the robot. These constraints are discussed in the following section.

Here, we demonstrate that although the CTR model in (1) is quasi-static, it has a time-dependent behavior. First, it is assumed that at a given time $t = t_k$, time-dependent variables are constant and the equations are solved in spatial domain (with respect to s). Next, the time-dependent variables are updated (i.e., $\theta^i(t)$, $\phi^i(t)$), and new transition points for separating the tubes are defined for the next sampling time, $t = t_{k+1}$. Finally, the equations are solved again in the spatial domain with the updated time-dependent variables. This process is repeated to calculate the robot's motion in the time domain.

To solve (1) in the spatial domain, a BVP must be solved as the curvatures of the tubes at the robot base along the z direction are

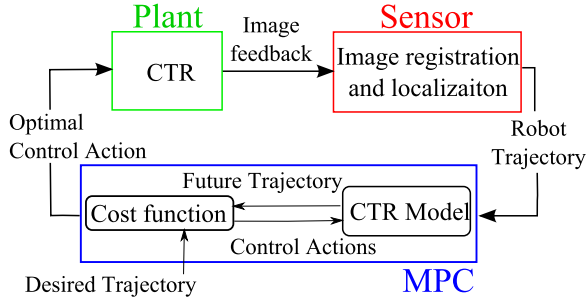


Fig. 3. Block diagram of MPC control loop.

unknown. Commonly, a shooting method is used to estimate the tubes base curvature $u_3^i(0, t_k)$ at a given time t_k as a function of the known boundary conditions at the robot tip $u_3^i(\ell^i + \phi^i(t), t_k)$, thus satisfying the tip boundary conditions. Based on (2d), satisfying the boundary conditions requires that the twist curvature of tubes tip is constant at all times, i.e.

$$\frac{\partial u_3^i(\ell^i + \phi^i(t), t)}{\partial t} = 0. \quad (3)$$

Here, we show that solving the BVP, which leads to (3), imposes nonholonomic constraints on the motion of the robot. The time variation of $\mathbf{R}^1(s, t)$ during robot motion has the form of

$$\dot{\mathbf{R}}^1(s, t) = \mathbf{R}^1(s, t)\hat{\omega}^1(s, t) \quad (4)$$

where $\hat{\omega}^1(s, t)$ is the first tube's angular velocity. Equating the mixed partial derivatives of $\mathbf{R}^1(s, t)$ by taking the derivative of (1b) with respect to t and derivative of (4) with respect to s gives

$$\begin{aligned} \mathbf{R}^{1'}(s, t)\hat{\omega}^1(s, t) + \mathbf{R}^1(s, t)\hat{\omega}^{1'}(s, t) \\ = \dot{\mathbf{R}}^1(s, t)\hat{\omega}^1(s, t) + \mathbf{R}^1(s, t)\dot{\hat{\omega}}^1(s, t). \end{aligned} \quad (5)$$

Substituting from (1b) and (4), premultiplying by $\mathbf{R}^{1T}(s, t)$, applying the identity $[a \hat{\times} b] = \hat{a} \hat{b} - \hat{b} \hat{a}$, and taking the $\{\cdot\}^\vee$ operator on the entire equation yields

$$\mathbf{u}^1(s, t) \times \omega^1(s, t) + \omega^{1'}(s, t) = \dot{\omega}^1(s, t). \quad (6)$$

Equating the time derivative of the first tube end curvature using the third vector component of (6) and (3) gives

$$\begin{aligned} u_2^1(\ell^1 + \phi^1, t) \omega_1^1(\ell^1 + \phi^1, t) \\ - u_1^1(\ell^1 + \phi^1, t) \omega_2^1(\ell^1 + \phi^1, t) + \omega_3^{1'}(\ell^1 + \phi^1, t) = 0. \end{aligned} \quad (7)$$

Equation (7) denotes the kinematic constraints on the motion of the robot end-effector due to robot tip boundary condition. It is nonholonomic and does not reduce the degrees of freedom of the robot. However, it reduces the robot's accessibility because the motion of the robot is confined to a particular surface defined by (7). The nonholonomic constraint implies dependence of CTR configuration to its path history, as shown experimentally in [14]. Therefore, controllers that take decisions based on contemporary information may lead the CTR to configurations that it cannot recover from. This issue is tackled by the proposed controller.

III. CONTROL SYSTEM DESIGN

We develop an MPC that can safely and precisely steer a CTR along predefined trajectories with low computational overheads and respect of workspace boundaries, joint constraints, and path dependence. Model predictive control theory [16] has never so far been investigated in the context of CTR. The controller's goal is to steer the robot tip along a desired trajectory. Fig. 3 depicts a block diagram of the developed closed-loop control algorithm, comprising the following.

- 1) A kinematic model of a CTR that predicts the robot trajectory given the robot tubes' rotation and translation as inputs.
- 2) A stereo camera system that estimates the robot's tip position.
- 3) The nonlinear MPC that manipulates the robot inputs to steer it along a desired trajectory.

The MPC controller calculates control decisions that iteratively minimize a cost function representing an error in desired versus current robot state parameters. We implement the robot's input saturations and boundary conditions of the kinematic models as equality, and inequality, constraints on the cost function, respectively. We now give a detailed mathematical description of the MPC scheme.

A. Model Predictive Control

The controller will be designed to track a time-varying trajectory, $\mathbf{x}_d(t)$. This trajectory is essentially a trajectory in state space, and can, therefore, include any parameter that can be considered robot state (e.g., stiffness, position, orientation, and energy). For practical reasons, we refer to $\mathbf{x}_d(t)$ as a trajectory in 3-D Cartesian space. Without loss of generality, the CTR end-effector is assumed to be the tip of the most inner tube, and is denoted by $\mathbf{x}(t) = \int_0^{\ell^1 + \phi^1(t)} \mathbf{r}^{1'}(s, t) ds$. First, let us summarize the solution of the CTR model given in (1) as

$$\mathbf{x} = f(t, \mathbf{y}(s, t)) \quad (8a)$$

$$\mathbf{y} = g(s, t, \mathbf{y}(s, t), \mathbf{q}(t)) \quad (8b)$$

where $\mathbf{y} = [\mathbf{u}^1(s, t) \dots \mathbf{u}^N(s, t)]$, and the actuator value vector $\mathbf{q}(t)$ consists of the rotations and translations of each tube, $\theta^i(t)$ and $\phi^i(t)$, as shown in Fig. 2. We note that despite the quasi-static nature of CTR model, t is included in (8) to reflect the path dependence of the CTR trajectory due to nonholonomic constraints.

To cast the kinematics problem into an IVP problem, the framework initially assumes that the curvatures of the tubes at the entry point $\mathbf{y}(0, t)$ are known. Then, (8) is the solution of (1) as an IVP. In later steps, these values are updated through an optimization process to respect BVP constraints. Also, it is assumed that there exists a control value $\mathbf{q}^*(t)$ that satisfies $\mathbf{x}^* = f(t, \mathbf{y}^*)$ and $\mathbf{y}^* = g(s, t, \mathbf{y}^*, \mathbf{q}^*)$. Here, $\mathbf{x}^*$ and $\mathbf{y}^*$ are the solution of (8) as an IVP with known $\mathbf{y}(0, t)$ as initial boundary conditions.

The proposed controller uses the CTR model to predict and optimize the future behavior of the system. The idea of the MPC is that given the last measurement of CTR end-effector position available at each sampled time instant, we optimize the predicted behavior of the system based on the CTR model, over a finite time horizon $\Gamma = t_0, \dots, t_k, \dots, t_{K-1}$ with $K \geq 2$. Only the first element of the predicted optimal input sequence is used as a feedback control value for the next sampling interval. Subsequently, the horizon is shifted one step forward and a new optimization problem is formulated and solved. This way the controller utilizes information about the future trajectory to select the proper control inputs at the present. This is a key feature of MPC that helps the controller avoid singularities in robot workspace and robot configurations that it cannot recover from while satisfying constraints on robot motion.

The MPC calculates the control decisions based on iterative optimization of the quadratic cost function

$$\mathcal{L}(\mathbf{x}_p, \mathbf{q}) = (\mathbf{x}_p(t) - \mathbf{x}_d(t)) \mathbf{W} (\mathbf{x}_p(t) - \mathbf{x}_d(t))^T \quad (9)$$

where $\mathbf{W}$ is a weighting matrix, $\mathbf{x}_d(t)$ the desired trajectory, and $\mathbf{x}_p(t)$ is the predicted future trajectory given an initial value of control input $\mathbf{q}_0$ and a control sequence $\mathbf{q}(t)$. The predicted trajectory $\mathbf{x}_p(t)$ is calculated iteratively based on (8) as

$$\mathbf{y}(t_{k+1}) = g(\mathbf{y}(t_k), \mathbf{q}(t_k)) \quad (10)$$

$$\mathbf{x}_p(t_{k+1}) = f(\mathbf{y}(t_{k+1})). \quad (11)$$

Given the cost function in (9) and a prediction horizon length K , the nonlinear MPC scheme is formulated as Algorithm 1. The appropriate

Algorithm 1: Nonlinear MPC Algorithm at Each Sampling Time t_k .

- 1 Measure the state of the system $x(t_k)$.
 - 2 Set $x(t_0) = x(t_k)$.
 - 3 Obtain the optimal control sequence $\mathbf{q}^*$ by solving:
Minimize: $\mathcal{J}(x(t_0), \mathbf{q}) := \int_0^{t_{K-1}} \mathcal{L}(\mathbf{x}_p, \mathbf{q}) dt$
with respect to: $\mathbf{q}$
subject to: $\begin{cases} \mathbf{y}(t_{k+1}) = g(\mathbf{y}(t_k), \mathbf{q}(t_k)), \\ \mathbf{x}_p(t_{k+1}) = f(\mathbf{y}(t_{k+1})). \end{cases}$
 - 4 Define the new feedback value $q(x(t_k)) := \mathbf{q}^*$ and use this control value in the next sampling period t_{k+1} .
-

prediction horizon for the MPC is estimated with experimentation to balance predictive capabilities (extended horizons preferred) with computational cost (reduced horizons preferred).

We now present a numerical discretization approach to the following:

- 1) to account for the effects of distal boundary conditions;
- 2) to include inequality constraints on control inputs;
- 3) to solve the nonlinear optimal control problem within the MPC algorithm.

B. Recursive Discretization of the MPC

We have so far neglected the boundary conditions at the end of the tubes in (2d) in the kinematic model used in the MPC. Here, we propose a discretization approach that allows implementation of the boundary conditions. It also transforms the MPC problem (Algorithm 1) into a standard nonlinear optimization problem, which can be solved via well-known robust nonlinear optimization algorithms. A standard nonlinear optimization problem has the form of

$$\begin{aligned} &\text{minimize: } \mathbf{F}(\mathbf{z}) \\ &\text{with respect to: } \mathbf{z} \\ &\text{subject to: } \mathcal{E}(\mathbf{z}) = 0, \mathcal{I}(\mathbf{z}) > 0 \end{aligned} \quad (12)$$

where $\mathcal{E}$ and $\mathcal{I}$ denote the equality and inequality constraints, respectively.

A common approach for transforming the MPC problem to (12) is the full discretization and incorporation of the robot model as constraints [17]. This method is straightforward but computationally inefficient as the optimization variables will include both the system states (e.g., curvatures along the length of the robot) and control inputs. Another approach is to decouple the control system from the optimization problem by recursively estimating the optimal solution $\mathbf{x}_p(t_{k+1})$ in (11) using the model of (8) outside of the optimization problem. This way, only the control inputs are optimization variables. However, because we are solving the IVP problem and neglecting the boundary conditions at the end of the tubes given in (2c), this method would result in a significant error. To overcome these problems, we implement a method known as shooting discretization [17].

The idea of shooting discretization, not to be confused with the shooting methodology employed to solve the BVP in, e.g., [1], is to include in the problem-selected components of the CTR states (i.e., initial curvature of the tubes), and kinematics (i.e., end curvature of the tubes), as independent optimization variables, and constraints, respectively. Proceeding this way, we provide useful information from the model to the optimizer throughout the optimization, and include the boundary conditions at the robot tip. The selected independent optimization variables are the curvatures of the tubes along the z direction at the entry point, i.e., $\psi(0, t) = [\mathbf{u}_3^1(0, t) \dots \mathbf{u}_3^N(0, t)]$. Thus, the new set of optimization variables $\mathbf{z}$ are

$$\mathbf{z} := \{\mathbf{q}(t_0), \psi(0, t_0), \dots, \mathbf{q}(t_{K-1}), \psi(0, t_{K-1})\} \quad (13)$$

Algorithm 2: Discrete Nonlinear MPC Algorithm at Each Sampling Time t_k .

- 1 Measure the state of the system $x(t_k)$.
 - 2 Set $x(t_0) = x(t_k)$.
 - 3 Obtain the optimal control sequence $\mathbf{q}^*$ by solving:
Minimize: $\mathbf{F}(\mathbf{z}) := \sum_{i=0}^{K-1} \mathcal{L}_i(t_i, \mathbf{x}_p, \mathbf{q})$
w.r.t.: $\mathbf{z} := \{\mathbf{q}(t_0), \psi(0, t_0), \dots, \mathbf{q}(t_{K-1}), \psi(0, t_{K-1})\}$
subject to: $\begin{cases} \mathcal{E}(\mathbf{z}) = 0, \\ \mathcal{I}(\mathbf{z}) > 0 \end{cases}$
 - 4 Define the new feedback value $q(x(t_k)) := \mathbf{q}^*$ and use this control value in the next sampling period t_{k+1} .
-

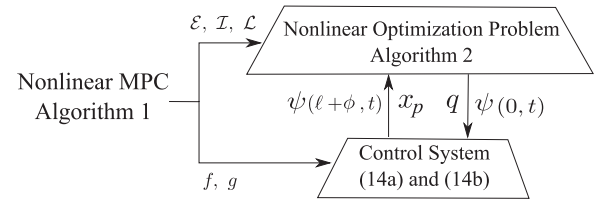


Fig. 4. Data communication between the elements of the control hierarchy.

where $\psi(0, t)$ are the unknown initial values of curvature. Afterward, we impose a set of constraints on the robot curvature (end point/boundary) to ensure the optimal solution satisfies the boundary conditions. The constraints on the control inputs and the kinematics of the model used in the optimization are

$$\mathcal{E}(\mathbf{z}) := \begin{cases} u_3^i(\ell^i + \phi^i, t_k) - U_3^i & (i = 1 \dots N, k = 1 \dots K) \\ \mathbf{x}_p(t_0) - \mathbf{x}(t_0) \end{cases} \quad (14a)$$

$$\mathcal{I}(\mathbf{z}) := \begin{cases} -\phi^N - \epsilon \\ -\phi^i + \phi^{i+1} - \epsilon & (i = 1 \dots N - 1) \\ \phi^N - \phi_{\max} \end{cases} \quad (14b)$$

The equality constraints $\mathcal{E}(\mathbf{z})$ are the desired boundary conditions at the distal end of the CTRs tubes. By satisfying these constraints throughout the optimization, the optimal solution of the IVP converges to the BVP. The term in the second row of $\mathcal{E}(\mathbf{z})$ ensures the optimal solution begins from the current position of the robot. Without this term, trajectory continuity through time would not be respected. The inequality constraints $\mathcal{I}(\mathbf{z})$ are the limits on the translational motion of the CTRs tubes. The desired linear offset between the tubes' base is ϵ , representing transmission/collar couplings, while $\phi_{\max}$ is the maximum allowed distance of the innermost tube from the entry point, $s = 0$. Implementing $\mathcal{I}(\mathbf{z})$ in the optimization ensures that the solution satisfies the mechanical limits of the robot and that the base of the tubes will not collide with each other or pass the entry point. It has also been shown that keeping the tubes' bases far from collision results in higher manipulability [18] and reduces the possibility of instability [6].

We can now define the new nonlinear MPC algorithm as standard nonlinear optimization (12) in Algorithm 2.

Fig. 4 shows the hierarchy within the control system based on Algorithm 2. A numerical solver tackles the underlying differential equation of the CTR model of (1) and (8) as IVP for given control sequences $\mathbf{q}$. These sequences correspond to values required by the nonlinear optimization solver in Algorithm 2. The interaction between these two components consists of sending control sequences $\mathbf{q}$ and initial boundary conditions $\psi(0, t)$ from the nonlinear optimal problem solver to the solver of the CTR model, which in turn sends computed state sequences $\mathbf{x}_p$ boundary values at the distal end of robot, $\psi(\ell + \phi, t) = [\mathbf{u}_3^1(\ell^1 + \phi^1, t) \dots \mathbf{u}_3^N(\ell^N + \phi^N, t)]$, back to the

nonlinear optimal problem solver. Next, a numerical solution to the nonlinear optimal problem listed within Algorithm 2 is presented.

C. Solution of the MPC

An interior-point method [19], efficient at handling equality and inequality constraints via appropriate implementation of *slack variables*, solves the nonlinear optimal control problem listed within Algorithm 2. In the interior-point method, a sequence of optimization variables, i.e., $\mathbf{z}$, that always belong to the feasible set of optimization variables that satisfy the constraints, is generated. For generating this sequence, in each iteration, the full set of inequality constraints is used via slack variables γ that transfer inequality constraints to active equality constraints with a nonnegativity constraint on the slack variable. To this end, the optimization problem is reformulated

$$\begin{aligned} &\text{minimize: } F(\mathbf{z}) \\ &\text{with respect to: } \mathbf{z}, \gamma \\ &\text{subject to: } \mathcal{E}(\mathbf{z}) = 0, \mathcal{I}(\mathbf{z}) - \gamma = 0, \text{ and } \gamma \geq 0. \end{aligned} \quad (15)$$

At this point, to include the effects of constraints, we replace the cost function in Algorithm 2 with the Lagrangian

$$L = F(\mathbf{z}) - \lambda_1^T \mathcal{E}(\mathbf{z}) - \lambda_2^T (\mathcal{I}(\mathbf{z}) - \gamma) \quad (16)$$

where λ_1 and λ_2 are Lagrange multipliers.

Now, a first-order necessary optimality condition commonly known as Karush–Kuhn–Tucker (KKT) can be applied to the Lagrangian to solve the optimization problem. Let us denote by Δ the gradient with respect to $\mathbf{z}$. Then, assuming that $\Delta_z(\mathcal{E})$ and $\Delta_z(\mathcal{I})$ are linearly independent, KKT provides a set of necessary conditions for a solution in nonlinear programming to be optimal. The minimization problem listed within Algorithm 2 indeed exhibits this property because the constraints given in (14a) and (14b) are linearly independent. The KKT conditions for (16) can be written as $\kappa = 0$, where κ is

$$\kappa = \begin{bmatrix} \Delta_z F(\mathbf{z}) - \Delta_z \mathcal{E}(\mathbf{z})^T \lambda_1 - \Delta_z (\mathcal{I}(\mathbf{z}))^T \lambda_2 \\ \lambda_2 - \mu \text{diag}(\gamma)^{-1} e \\ \mathcal{E}(\mathbf{z}) \\ \mathcal{I}(\mathbf{z}) - \gamma \end{bmatrix} \quad (17)$$

with $\gamma > 0$, $\lambda_2 > 0$, $e = [1, 1, \dots, 1]^T$, and μ is an initially positive integer that acts as a perturbation parameter added to the equations. The interior point method consists of solving the perturbed KKT conditions ($\kappa = 0$) for a sequence of positive μ that converges to zero [19]. The strict positivity of μ in each iteration forces the slack variable γ and the multiplier λ_2 to be positive, and therefore, the solution to stay away from the boundary of the feasible set of solutions defined by the constraints. When $\mu \rightarrow 0$, the optimal solution of (16) will be independent of slack variables while satisfying the inequality constraints. Newton's method to solve (17) gives

$$\begin{bmatrix} \Delta_{zz}^2 L + \sigma_1 \mathbf{I} & \mathbf{0} & \Delta_z \mathcal{E}^T & \Delta_z \mathcal{I}^T \\ \mathbf{0} & \Sigma & \mathbf{0} & -\mathbf{I} \\ \Delta_z \mathcal{E} & \mathbf{0} & -\sigma_2 \mathbf{I} & \mathbf{0} \\ \Delta_z \mathcal{I} & -\mathbf{I} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \delta z \\ \delta \gamma \\ -\delta \lambda_1 \\ -\delta \lambda_2 \end{bmatrix} = \kappa \quad (18)$$

where $\Sigma = \text{diag}(\gamma)^{-1} \text{diag}(\lambda_2)$, and $\mathbf{I}$, and $\mathbf{0}$, are the identity, and zero matrices of appropriate dimensions, respectively. Variables $\sigma_1 > 0$

Algorithm 3: Solver of Nonlinear Optimal Problem.

```

1 Suppose  $(\mathbf{z}, \gamma) := (\mathbf{z}_0, \gamma_0)$  (arbitrary) and set  $m := 0$ .
2 Compute multipliers  $\lambda_{10}$  and  $\lambda_{20}$  from (17) and define
   parameters  $\mu_0 > 0$  and  $\varepsilon \in (0, 1)$ .
3 do
4   do
5     Compute search direction  $\delta$  via (18),
6     Find  $\Theta := \max \{ \Theta \in (0, 1) \mid \gamma + \Theta \delta \gamma > 0 \}$ ,
7     Set  $\mathbf{z}_{m+1} := \mathbf{z}_m + \Theta \delta \mathbf{z}$ ,
       Set  $\gamma_{m+1} := \gamma_m + \Theta \delta \gamma$ ,
       Set  $\lambda_{1m+1} := \lambda_{1m} + \delta \lambda_1$ ,
       Set  $\lambda_{2m+1} := \lambda_{2m} + \delta \lambda_2$ ,
8     Set  $\mu_{m+1} := \mu_m$ ,
       Set  $m := m + 1$ .
       Compute  $\mathcal{C} := \|\kappa\|_\infty$ .
9   while  $\mathcal{C} > \mu_m$ ;
10  Choose  $\mu_m \in (0, \varepsilon \mu_m)$ .
11 while  $\mathcal{C} > \varepsilon_{\text{Tot}}$ ;
```

TABLE I
PHYSICAL PARAMETERS FOR CTRs TUBES

	Inner Radius [mm]	Outer Radius [mm]	Straight Length [mm]	Curved Length [mm]	U [m ⁻¹]	E [GPa]	G [GPa]
1 st Tube	0.35	0.55	431	103	21.3	10.25	18.79
2 nd Tube	0.7	0.9	330	113	13.1	68.6	11.53
3 rd Tube	1	1.2	174	134	3.5	16.96	14.25

TABLE II
RESULTS OF 3-D POINT TRACKING

e_{mean} [mm]	PO	t_r [sec]	t_s [sec]	σ [mm]
0.5	7%	0.64	0.85	0.70

and $\sigma_2 > 0$ are damping factors introduced to ensure the positive definiteness of the Hessian matrix $\Delta_{zz}^2 L$, and to avoid rank deficiency of $\Delta_z \mathcal{E}$, respectively. By solving (18), the step sizes for estimation optimization variables δ can be obtained. Finally, the MPC problem is solved following Algorithm 3.

In Algorithm 3, Θ is estimated to ensure slack variables neither become zero nor reach their lower bounds too fast. The error function $\mathcal{C}$ is the infinity norm of κ , and is implemented as termination criterion and measure of convergence and fulfillment of KKT optimality condition. Algorithm 3 can be used to solve the MPC problem at each sampling time. In the next section, simulations are performed to evaluate the proposed MPC controller.

IV. MPC EVALUATION VIA SIMULATIONS

Simulations compare MPC performance with Jacobian-based controllers, which to the best of authors knowledge, are the most common type of controller for CTRs [8], [9], [20]. The Jacobian can be used with the following closed-loop control law to steer the CTR [21]:

$$\dot{\mathbf{q}}_d = \mathbf{J}^\dagger [\dot{\mathbf{x}}_d + \mathbf{K}_p (\mathbf{x}_d - \mathbf{x})] \quad (19)$$

$$\text{RMSE}_u = \sqrt{\frac{\sum_{i=1}^n \left(\left\| [U_3^1, U_3^2, U_3^3]^T \right\| - \left\| [u_3^1(\ell^1 + \phi^1, t_n), u_3^2(\ell^2 + \phi^2, t_n), u_3^3(\ell^3 + \phi^3, t_n)]^T \right\| \right)^2}{n}} \quad (20)$$

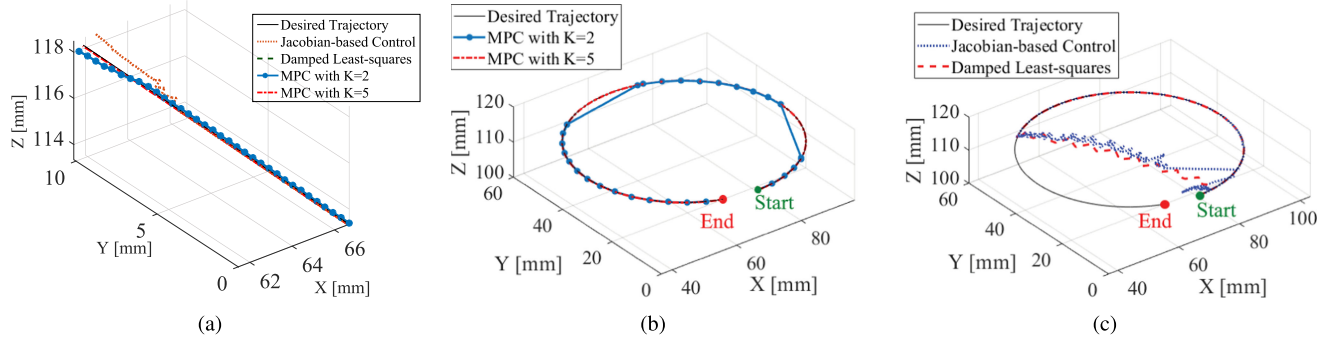


Fig. 5. Comparison between Jacobian-based controllers, MPC controller with horizon of 2, and MPC controller with horizon of 5. (a) Linear trajectory tracking. (b) MPC controller for circular trajectory tracking. (c) Jacobian-based controllers for circular trajectory tracking.

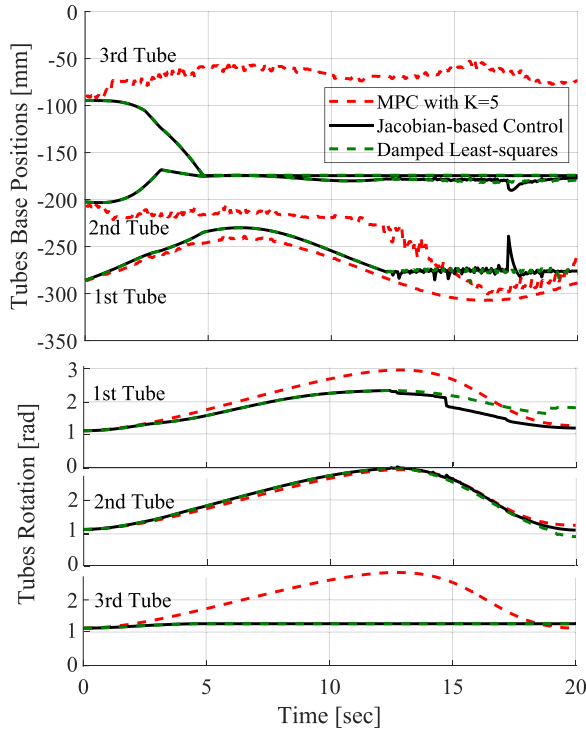


Fig. 6. Comparison between position of the base of robot's tubes and rotation of the tubes for Jacobian-based controllers and MPC controller.

where $\mathbf{J}^\dagger$ is the pseudoinverse of the robot Jacobian, $\mathbf{K}_p$ is a symmetric positive definite matrix, $\mathbf{q}_d$ is the control input, and $\mathbf{x}_d$ is the desired robot trajectory in its task space. An effective strategy that allows Jacobian-based control of robots in the neighborhood of kinematic singularities is the damped least-squares technique. The method improves inverse kinematics when the Jacobian is ill-conditioned and close to a singularity by adding a damping factor [21]. In this case, the pseudoinverse of the robot Jacobian in (19) can be replaced by $(\mathbf{J}^T \mathbf{J} + \Lambda^2 \mathbf{I})^{-1} \mathbf{J}^T$, where Λ is the damping factor. Here, we compare the performance of the MPC controller to one that uses the control law of (19) with and without damping.

The controllers are tested on a perturbed CTR model with $\pm 10\%$ uncertainty in the values of the tubes' Young and shear moduli with respect to the nominal ones. The selected robot is purposefully unstable within its workspace to test the performance of the controller in avoiding instabilities. The CTR design parameters are shown in Table I. For consistency, these parameters are identical to those used in our experimental setup in Section V. Simulations show that local

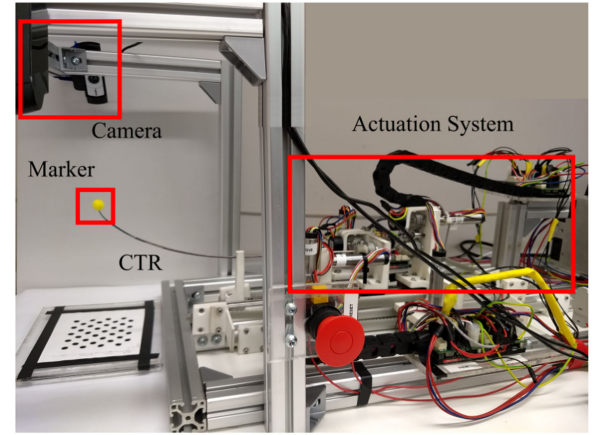


Fig. 7. CTR comprises three precurved Nitinol tubes. The robot uses six motors (Maxon, CH), connected to pulleys and timing belts for rotating and translating the tubes. Motor position controllers (EPOS4 Compact 50/5 CAN, Maxon, CH) are connected to a PC using a CAN-to-USB interface (Kvaser Inc., CA, USA). A stereo rig tracks the robot tip in 3-D.

controllers (e.g., Jacobian based) blindly steer the robot to configurations it cannot recover from, whereas the proposed MPC incorporates future knowledge to follow optimal paths. We performed a simulation study to select the appropriate prediction horizon. We incremented the horizon of the controller from 2 to 10 and used it to track several linear and circular trajectories. Increasing the prediction horizon to higher than 5 did not significantly improve the performance of the controller, whereas it precluded our desired sampling time of 150 ms on the robot workstation (nonreal-time performance). Therefore, we decided to use an MPC with prediction horizon of 5 that satisfies both the performance and real-time criteria. Results for prediction horizons of 2 and 5 are shown in Fig. 5(a) and (b). The controller parameters used in the simulations were $\mu = 0.1$, $\sigma_1 = \sigma_2 = 0.1$, and $C = 10^{-7}$. $\mathbf{K}$ in (19), and Λ , were selected as $2\mathbf{I}$, and 0.45, respectively, as these values were found to achieve the minimum tracking error. The simulation results for tracking a linear trajectory are shown in Fig. 5(a). The mean error of the Jacobian-based controller, and MPC controllers with prediction horizons of 2, and 5, are 0.203, 0.123, and 0.080 mm, respectively. The damped least-squares method had the same results as the Jacobian-based controller. All controllers satisfied the robot boundary conditions at its tip with a minimum accuracy of $5 \times 10^{-4} \text{ m}^{-1}$. Overall, the MPC with expanded horizon reduced the trajectory-following error by 80%.

In the second simulation study, a circular trajectory near the limits of the robot workspace was selected. Results of the robot tip trajectory in task space are shown in Fig. 5(b) and (c), whereas the evolution of the tubes' base position and base rotation values is shown in Fig. 6.

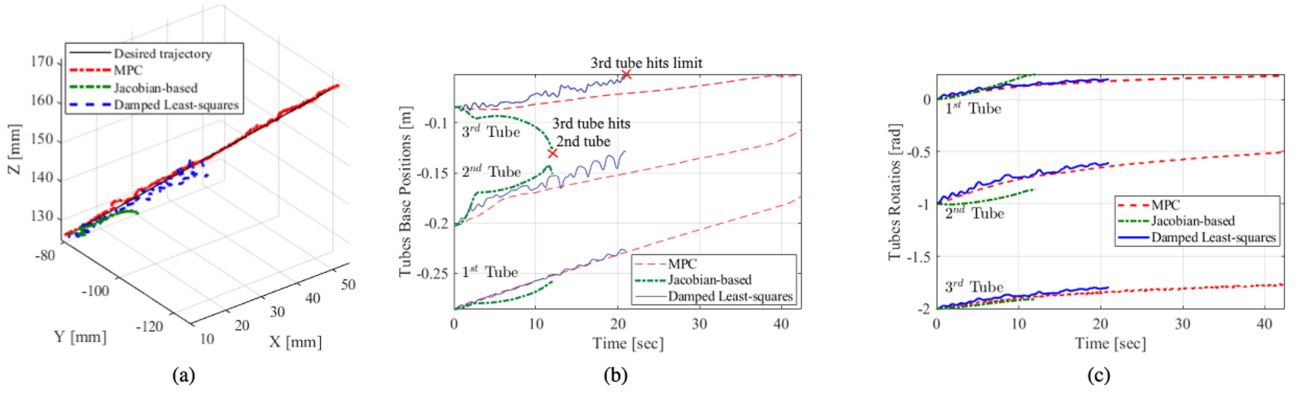


Fig. 8. Experimental comparison of the MPC against a controller with damped least-squares inverse kinematics. (a) Desired trajectory and actual trajectory. (b) Position base of robot's tubes, violation of joints' limits are shown with a red cross. (c) Angular position of base of robot's tubes.

Both Jacobian-based controllers initially follow the desired trajectory at the cost of failing the joints' constraints as seen in Fig. 6. In Fig. 6, snapping manifests when tube 3 is fully retracted and tube 2 is inserted up to its limit. At the same time, the difference between the rotation angle of tube 1 and tube 2 is close to 180° , which leads to an almost straight robot (see Fig. 1), instability, and subsequent snapping to a position of less energy. The damped least-squares controller slightly outperforms the simple Jacobian-based controller (e.g., see $T = 17$ s). The MPC with the short prediction horizon of 2 is also unable to find the optimal solution near the robot singularities, and simply skips those areas. The MPC with longer prediction horizon, however, not only satisfies joint constraints (see Fig. 6) but also steers the CTR on the desired trajectory and avoids workspace singularities.

V. EXPERIMENTAL EVALUATION AND DISCUSSION

Fig. 7 shows the experimental setup. Commonly, electromagnetic trackers are used to track the robot tip. Here, we simulated a scenario in which the same measurement is acquired using a calibrated stereo rig comprising two Logitech HD Pro C922 webcams. The cameras were running at 1080p resolution and 30 frames per second. As identified through calibration using on average 30 views of a circular calibration pattern, a single pixel corresponded to 0.2×0.2 mm on the image plane. A spherical colored marker was fixed at the tip of the CTR. The marker was detected by color thresholding, and its triangulated 3-D position was used as the MPC input.

Manual backbone segmentation established the base and shape of the CTR relative to the aligned calibration grid. Matching backbone points were selected in both images, and then triangulated to provide the 3-D point cloud. The extracted 3-D backbones were used to calibrate for the CTR model parameters, namely, Young's and shear moduli of the tubes. The parameters were identified by fitting the kinematic model given in (1) to the shape of the robot estimated via the cameras at 25 different configurations. The identified parameters of the model are given in Table I, and are the same as those used in the simulations. Tube diameters were selected to ensure smooth telescopic translation of the tubes with minimal friction. The maximum error of the model in predicting the CTR tip position was 1.74 cm, corresponding to 9% of robot's length, which is inline with [2], where the model was proposed.

The identified model was implemented in the MPC to steer the CTR. Other parameters of the controller were practically identified as $\mu = 0.1$, $\sigma_1 = \sigma_2 = 0.1$, and $C = 10^{-7}$. An Intel Core i7 (2.93 GHz) machine was used to solve the optimal control problem at sampling time of 150 ms. The prediction horizon of the MPC, K , was equal to 5, as per our simulations. To minimize the effect of delays caused by image processing, two computers connected via UDP communication were used to control the robot. One tracked the robot tip using ROS (Linux

OS), and one run the control algorithm using MATLAB Simulink Real-time (Windows OS). The measured round-trip time in the control-loop was 300 ms (i.e., twice the sampling time).

We evaluated the performance of the controller in reaching 3-D points, and following linear and circular trajectories. These are elemental trajectories that can make up any complex trajectory in 3-D space, and provide quantitative insights on controller performance.

In the first experiment, we evaluated the performance of the MPC in tracking single points across the robot workspace (step responses). We performed 20 (random) point tracking experiments, each repeated 4 times, in which the robot is tasked to move the chosen point in 3-D space. Mean error e_{mean} , percentage overshoot PO, response time t_r , settling time t_s , and standard deviation σ for the experiments are reported in Table II. The mean error of the MPC as a percentage of robot length surpasses the state-of-the-art (0.03% versus 0.05% in [10] and 0.09% in [9]; note that our three-tube robot is unstable, whereas the compared controllers were evaluated on a two-tube stable robot).

Next, we compared the performance of the controller with implemented controllers that use the inverse-Jacobian and damped least-squares inverse kinematics. The CTR was steered on a 3-D trajectory near its joint limits; ten trials were performed. A comparison between the MPC and damped least-squares inverse kinematics is shown in Fig. 8. It can be seen that the damped least-squares controller is unable to track the desired trajectory, whereas the joint constraints defined in (14a) are violated (the third tube reaches the entry point). However, the proposed MPC controller respects the joint limits until it reaches the robot's workspace boundaries. The limit is shown by a red cross in Fig. 8(b). The mean error and standard deviation of the controllers are compared in Table IV.

In the next set of experiments, we used the MPC to steer the CTR on 2-D and 3-D linear trajectories, and 2-D and 3-D circular trajectories. The trajectories were randomly selected across the robot workspace with the constraint to be near instabilities and workspace boundaries; 15 trials were performed for each scenario. Representative experimental results for 2-D circular trajectories, 3-D circular trajectories, and linear trajectories are shown in Fig. 9. The Jacobian-based controllers (both with and without damped least squares) failed to perform circular trajectory tracking. Results of all the experiments are summarized in Table III. Maximum of norm of error for trajectory tracking $e_{x \text{ max}}$, the root-mean-squared tracking error RMSE, maximum of norm of error for equality constraints in (14a) $e_{u \text{ max}}$, and root-mean-squared error of equality constraints RMSE_u are listed. The root-mean-squared error is

calculated as $\text{RMSE}_x = \sqrt{\frac{\sum_{i=1}^n (\|\hat{x}_i - \|x_i\|)^2}{n}}$, and is used as a measure of the differences between the desired value $\hat{x}$ and the actual value of the parameters experimentally observed, i.e., x for n data points. $e_{u \text{ max}}$ is calculated as $\max\{\sum_{i=1}^3 \|u_3^i(\ell^i + \phi^i, t_k) - U_3^i\|\}$. RMSE of curvature is calculated as shown in (20) shown at the bottom of page no. 5, and is used as a measure of the differences between the desired value of end

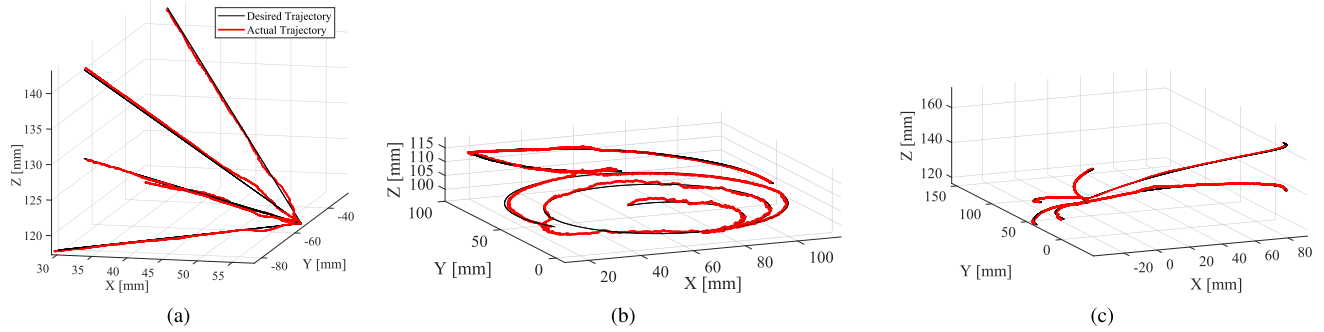


Fig. 9. Representative experimental results for trajectory tracking. (a) 2-D and 3-D linear trajectory tracking. (b) and (c) 2-D and 3-D circular trajectory tracking.

TABLE III
RESULTS OF TRAJECTORY TRACKING EXPERIMENTS

	Linear trajectory	Circular trajectory
$e_{x \max}$ [mm]	2.9	14
RMSE_x [mm]	1.2	1.7
$e_{u \max}$ [m^{-1}]	1.6×10^{-3}	1.4×10^{-3}
RMSE_u 10^{-4}	0.8×10^{-3}	2.2×10^{-3}

TABLE IV
COMPARISON OF JACOBIAN-BASED AND MPC CONTROLLERS

	MPC	Jacobian-based	Damped least-squares
RMSE [mm]	0.8	5.3	4.1
σ [mm]	0.5	3.1	2.4

curvature that satisfies the constraints in (14a), and the actual value of the end curvature obtained by the MPC for n data points.

The CTR is able to follow the trajectories with a maximum RMSE of 1.7 mm, corresponding to 1% of robot arclength. The maximum tip error occurs for one of the circular trajectories. Errors are due to the nonlinearity of CTRs kinematics compared to classic robots and implicit constraints on the motion of the robot as discussed in Sections I and II. The MPC reduces the errors and leads the robot tip back to the desired trajectory despite locally exhibiting its maximum error. Moreover, the controller satisfies the equality constraints on robot curvature, i.e., boundary conditions, with maximum error of $2.2 \times 10^{-3} \text{ m}^{-1}$.

VI. CONCLUSION

In this article, we introduced a tailored MPC for autonomous steering of CTR. The MPC approach was selected as it explicitly considers the constraints on the robot's workspace and boundary conditions. More importantly, by considering a time horizon, the MPC makes informed control decisions that steer the CTR away from configurations that it cannot recover from, e.g., joint limits, instabilities, and singularities. The MPC calculates control actions based on iterative optimization of the predictions of the kinematic model of CTR. Simulations and experiments showed that the proposed controller outperforms local Jacobian-based controllers by a large margin. Ongoing work focused on improving computational efficiency.

REFERENCES

- [1] P. E. Dupont, J. Lock, B. Itkowitz, and E. Butler, "Design and control of concentric-tube robots," *IEEE Trans. Robot.*, vol. 26, no. 2, pp. 209–225, Apr. 2010.
- [2] D. C. Rucker, B. A. Jones, and R. J. Webster, "A geometrically exact model for externally loaded concentric-tube continuum robots," *IEEE Trans. Robot.*, vol. 26, no. 5, pp. 769–780, Oct. 2010.
- [3] H. B. Gilbert, J. Neimat, and R. J. Webster, "Concentric tube robots as steerable needles: Achieving follow-the-leader deployment," *IEEE Trans. Robot.*, vol. 31, no. 2, pp. 246–258, Apr. 2015.
- [4] A. H. Gosline *et al.*, "Percutaneous intracardiac beating-heart surgery using metal MEMS tissue approximation tools," *Int. J. Robot. Res.*, vol. 31, no. 9, pp. 1081–1093, 2012.
- [5] C. R. Mitchell, R. J. Hendrick, R. J. Webster, and S. D. Herrell, "Toward improving transurethral prostate surgery: Development and initial experiments with a prototype concentric tube robotic platform," *J. Endourology*, vol. 30, no. 6, pp. 692–696, 2016.
- [6] C. Bergeles, A. H. Gosline, N. V. Vasilyev, P. J. Codd, P. J. del Nido, and P. E. Dupont, "Concentric tube robot design and optimization based on task and anatomical constraints," *IEEE Trans. Robot.*, vol. 31, no. 1, pp. 67–84, Feb. 2015.
- [7] K. Leibrandt, C. Bergeles, and G. Yang, "Concentric tube robots: Rapid, stable path-planning and guidance for surgical use," *IEEE Robot. Autom. Mag.*, vol. 24, no. 2, pp. 42–53, Jun. 2017.
- [8] D. C. Rucker and R. J. Webster, "Computing Jacobians and compliance matrices for externally loaded continuum robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2011, pp. 945–950.
- [9] R. Xu, A. Asadian, S. F. Atashzar, and R. V. Patel, "Real-time trajectory tracking for externally loaded concentric-tube robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2014, pp. 4374–4379.
- [10] K. Wu *et al.*, "Safety-enhanced model-free visual servoing for continuum tubular robots through singularity avoidance in confined environments," *IEEE Access*, vol. 7, pp. 21539–21558, 2019.
- [11] A. V. Kudryavtsev *et al.*, "Eye-in-hand visual servoing of concentric tube robots," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2315–2321, Jul. 2018.
- [12] M. Khadem, J. O'Neill, Z. Mitros, L. da Cruz, and C. Bergeles, "Autonomous steering of concentric tube robots for enhanced force/velocity manipulability," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 2197–2204.
- [13] M. M. Marinho, B. V. Adorno, K. Harada, and M. Mitsuishi, "Dynamic active constraints for surgical robots using vector-field inequalities," *IEEE Trans. Robot.*, vol. 35, no. 5, pp. 1166–1185, Oct. 2019.
- [14] J. Ha, G. Fagogenis, and P. Dupont, "Effect of path history on concentric tube robot model calibration," in *Proc. Hamlyn Symp. Med. Robot.*, 2017, pp. 77–78.
- [15] J. Ichnowski and R. Alterovitz, "Motion planning templates: A motion planning framework for robots with low-power CPUs," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2019, pp. 612–618.
- [16] R. R. Negenborn and J. M. Maestre, "Distributed model predictive control: An overview and roadmap of future research opportunities," *IEEE Control Syst. Mag.*, vol. 34, no. 4, pp. 87–97, Aug. 2014.
- [17] H. Bock and K. Plitt, "A multiple shooting algorithm for direct solution of optimal control problems," *IFAC Proc. Vol.*, vol. 17, no. 2, pp. 1603–1608, 1984.
- [18] M. Khadem, L. Da Cruz, and C. Bergeles, "Force/velocity manipulability analysis for 3D continuum robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 4920–4926.
- [19] J. Nocedal and S. Wright, *Interior-Point Methods for Nonlinear Programming*. New York, NY, USA: Springer, 2006, pp. 563–597.
- [20] J. Burgner *et al.*, "A telerobotic system for transnasal surgery," *IEEE/ASME Trans. Mechatronics*, vol. 19, no. 3, pp. 996–1006, Jun. 2014.
- [21] S. Chiaverini, B. Siciliano, and O. Egeland, "Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator," *IEEE Trans. Control Syst. Technol.*, vol. 2, no. 2, pp. 123–134, Jun. 1994.